

Оценка эффективности игроков в регби через event-based статистику матчей

ClickHouse для инженеров и
архитекторов БД



**Меня хорошо видно
& слышно?**



Защита проекта

Тема: Оценка эффективности игроков в регби через event-based статистику матчей



Андрушенко Дмитрий

Системный архитектор

УТИС АЙ ТИ Групп

Разработка программного
обеспечения позиционирования
шахтеров в подземных выработках

План защиты

Цель и задачи проекта

Какие технологии использовались

Что получилось (демонстрация проекта)

Выводы

Вопросы и рекомендации

Цель проекта

Показать, как аналитический отдел регби клуба может хранить каждое действие игрока на поле и автоматически считать показатели эффективности, а также как развернуть для этого инфраструктуру с визуализацией и мониторингом.

Задачи:

- хранить **event-log** — одна строка = одно действие (игрок, тип, координаты, исход, время);
- считать **метрики эффективности** по игрокам и позициям;
- давать **наглядные отчёты** тренеру и аналитику.

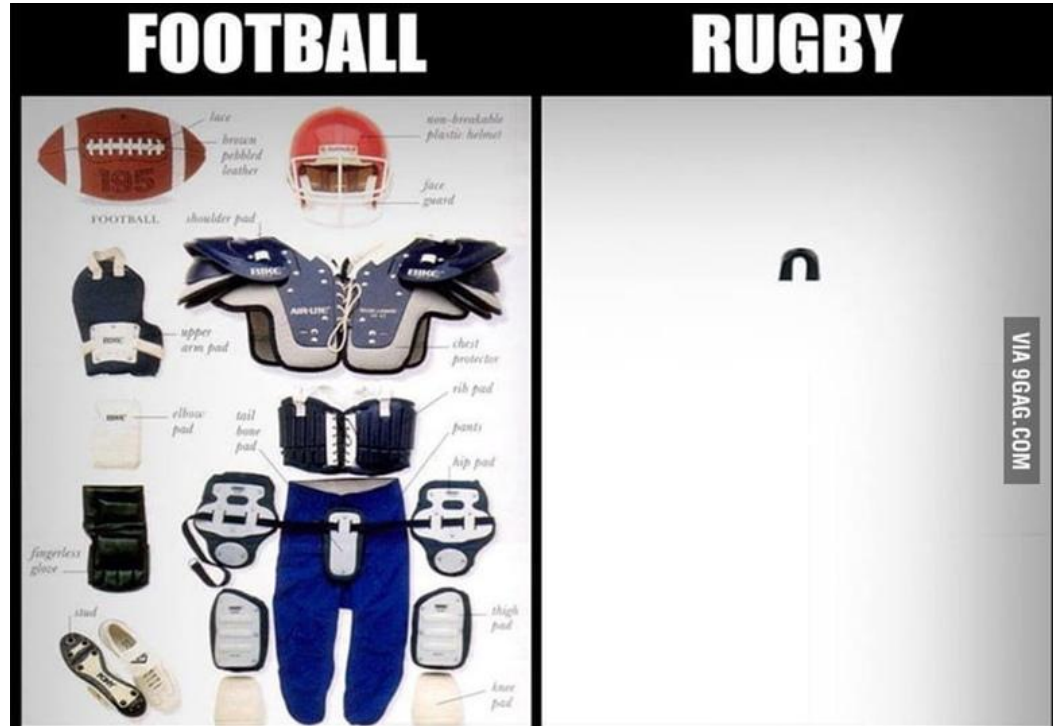


Регби: что это и почему это не американский футбол

- Регби — это командная контактная игра, где мяч заносят в зачетную зону соперника.
- В регби нельзя передавать мяч вперед руками: пас разрешен только назад или вбок.
- Это не американский футбол: в регби 15 игроков на поле, игра длится 80 минут и идет почти без частых остановок и с меньшим количеством защитной экипировки.
- Мяч в регби и американском футболе имеет схожую овальную форму. Мяч регбистов менее вытянут, и при этом немного больше и тяжелее.



Регби: что это и почему это не американский футбол



Три метрики. 1 - Gain Line Success

Первое — Gain Line Success, процент успешных переносов мяча. Gain line — условная линия, где закончилась предыдущая фаза атаки. Если игрок с мячом продвинул игру за эту линию — это успех. Считаем только события типа `carry` с полем `gain\_line`.

Коротко: Продвигает ли игрок атаку, когда несёт мяч?

Формула

$$\text{Gain Line Success \%} = (\text{число carry с gain_line} = +1) / (\text{число carry с gain_line} = +1, 0 \text{ или } -1) \times 100$$

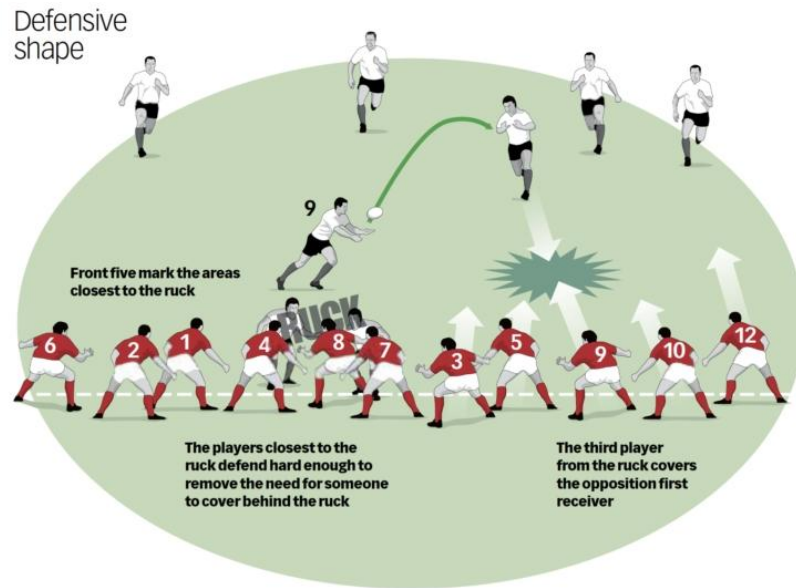
Числитель: переносы с успехом.

Знаменатель: все переносы, где мы явно оценили исход относительно линии (не сбрасываем «пустые» carries).

Пример:

10 переносов: 6 раз `+1`, 2 раза `0`, 2 раза `-1`.

Успех = $6 / 10 = 60\%$.



Три метрики. 2 - Offload Success

Второе — **Offload Success**, **Оффлоад** — это когда игрок, которого уже схватили (или схватывают), **всё равно отдаёт мяч своему партнёру**, чтобы атака не остановилась.

Offload Success % — как часто такие передачи **доходят до своего**, а не становятся потерей.

Важно: **много оффлоадов** $\neq$ **хорошие оффлоады**. Игрок с 20 попытками и 10 успехами (50%) хуже, чем с 8 попытками и 8 успехами (100%), если сравнивать только «объём».

Формула

$$\text{Offload Success \%} = (\text{offload с outcome} = \text{'success'}) / (\text{все offload}) \times 100$$

Пример:

12 оффлоадов, 9 успешных $\rightarrow 9 / 12 = 75\%$.



Три метрики. 3 - Tackle Completion

В защите игрок пытается **схватить** соперника с мячом (захват). **Tackle Completion %** показывает, **как часто отбор удаётся** — соперник действительно остановлен по правилам, а не ушёл из захвата.

Формула

$$\text{Tackle Completion \%} = (\text{tackle с outcome} = \text{'success'}) / (\text{все tackle}) \times 100$$

Пример:

50 попыток отбора, 42 успеха $\rightarrow 42 / 50 = 84\%$.



Тепловая карта. Heatmap

Три KPI — качество действий; heatmap — территория и вовлечённость по третям поля относительно своих ворот.

Назначение

Ответ на вопрос: **в какой трети поля (свои 22 / середина / чужие 22) игрок чаще совершает действия** — для тактики и скаутинга.

Цепочка данных

match_events (x_coord, team_id, player_id, event_type)



v_player_zone_activity

— field_zone = f(x_coord, team_id)

— events_count = count() по группе



Superset heatmap_v2: SUM(events_count)

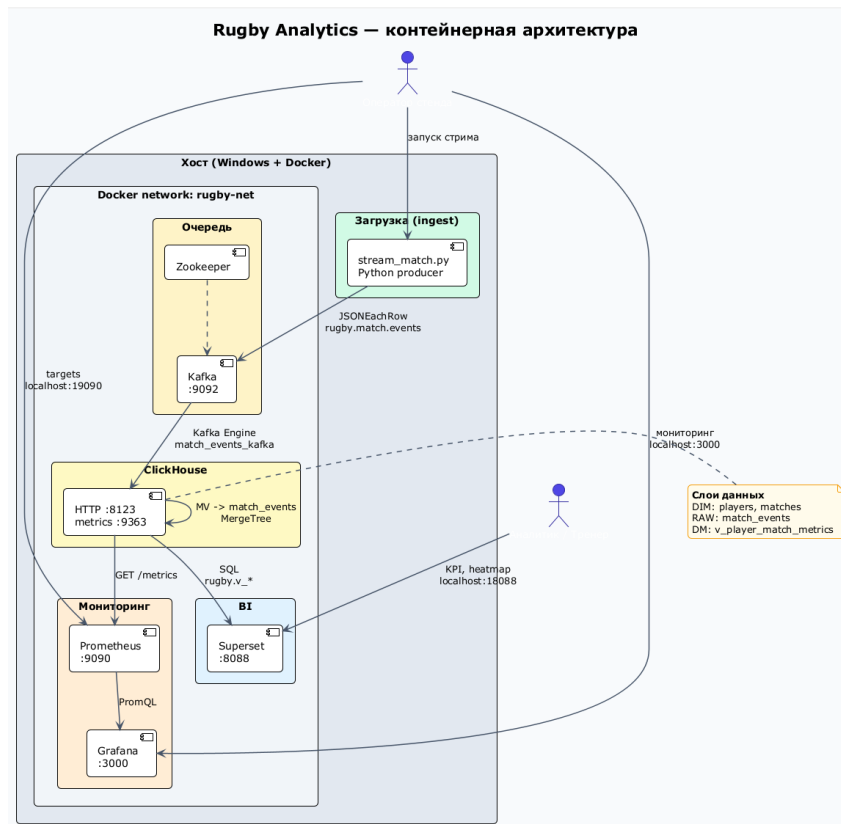
— строки: field_zone

— столбцы: player_name

— фильтр дашборда: team_name



Архитектура



Стек технологий

1. ClickHouse
2. DockerCompose
3. Python
4. Apache Kafka
5. Prometheus
6. Grafana
7. Apache Superset



Почему ClickHouse

- **Колоночное OLAP** — быстрые `GROUP BY` и `countIf` по миллионам событий.
- **Kafka Engine + Materialized View** — загрузка без отдельного ETL-сервиса.
- **Append-only event-log** — события не обновляем, только добавляем.
- **PostgreSQL** — хорош для состава и расписания, но не для сырого event-log на больших объёмах.



Поток одного события

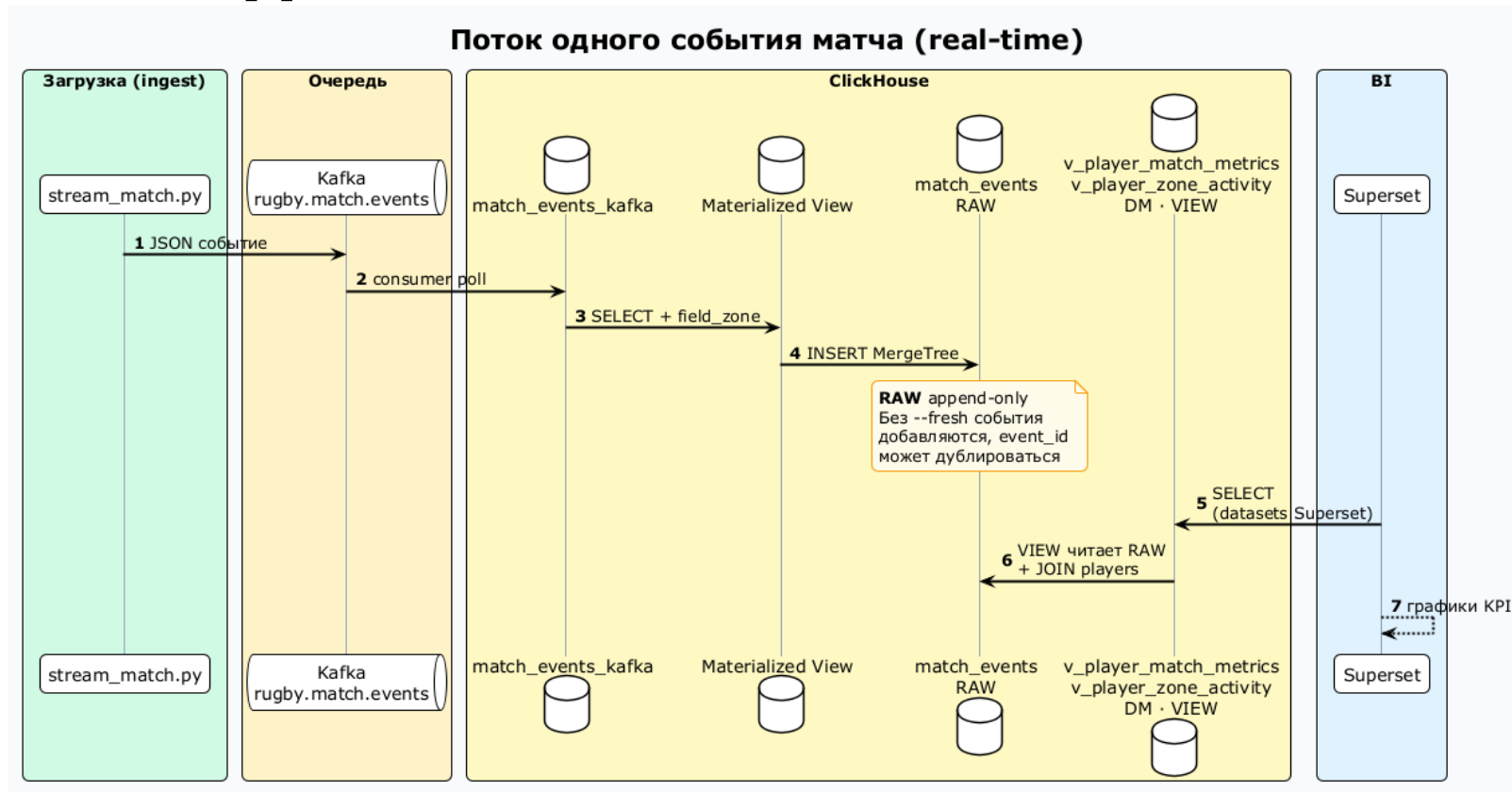
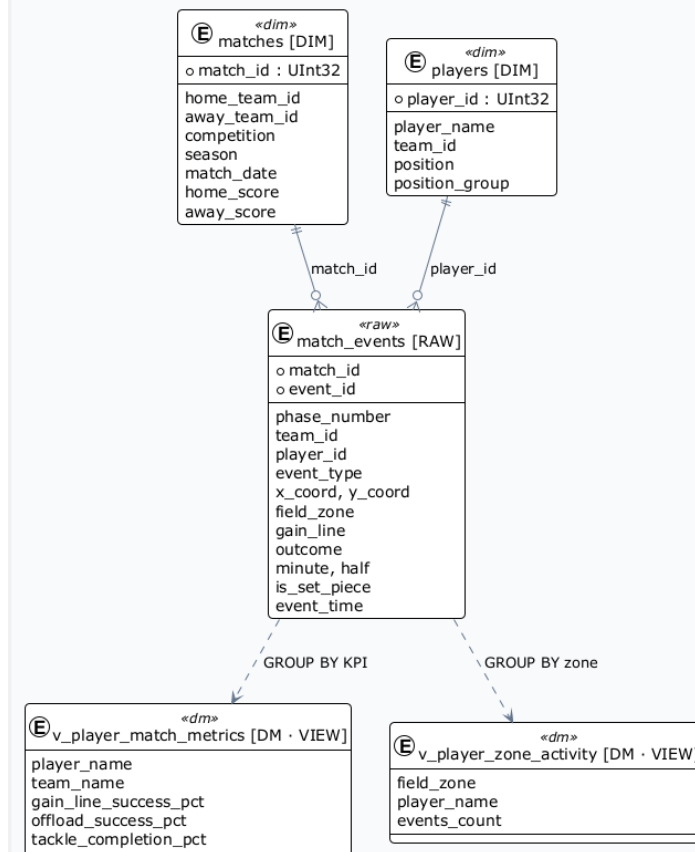


Схема данных

Логическая модель данных (БД rugby)



**Теперь давайте посмотрим на
проект в живую**



Выводы

Цели проекта достигнуты!

1. **Event-log** — таблица `match\_events`, поток из Kafka, одна строка = одно событие с координатами и исходом.
2. **Метрики эффективности** — три зафиксированных KPI в SQL-VIEW.
3. **Отчёты для тренера** — Superset: столбцы по игрокам, сводная таблица, фильтр по команде; heatmap отвечает на «где на поле действовал» — это часть той же цели «по событиям».



Вопросы и рекомендации



если есть вопросы

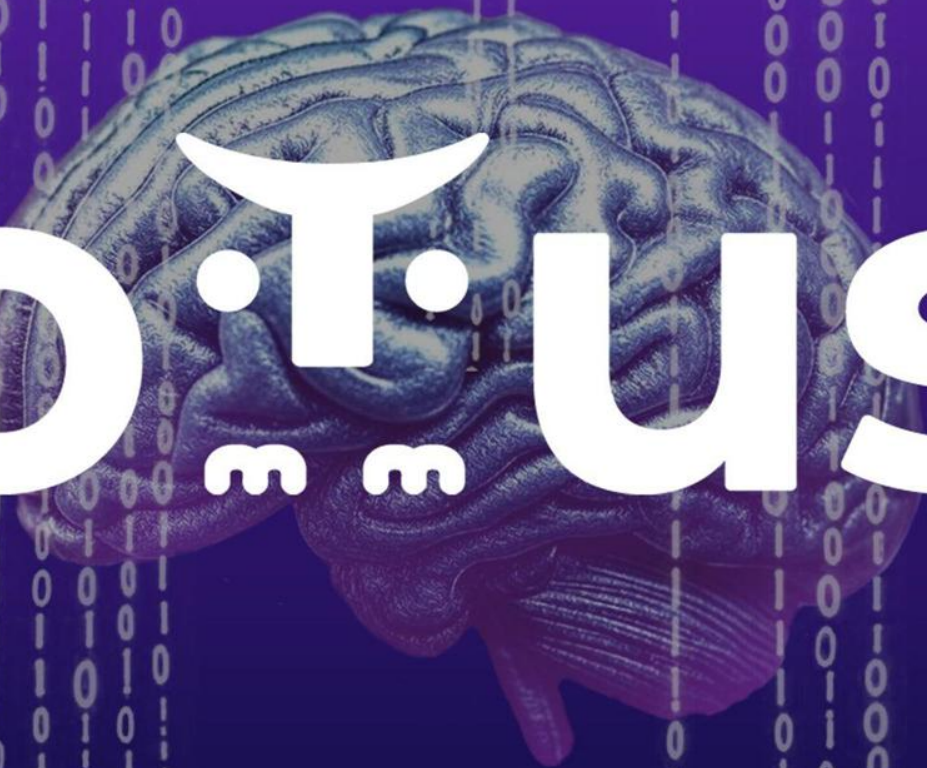


если вопросов нет



Спасибо за внимание!

OTUS





Инструменты для работы с презентацией

1. Текст:

- «Одна мысль — один слайд»
- Размещайте на каждом слайде до 5 строк текста, не более 5 слов в каждой строке
- Выделяйте главные идеи с помощью размера и цвета шрифта
- Используйте подзаголовки, если заголовок слишком длинный
- Группируйте информацию в визуальные блоки
- Делите большие текстовые блоки на колонки или списки
- Используйте стрелки для обозначения последовательности информации
- Чередуйте текстовые блоки с графическими элементами

2. Используйте шрифт Roboto, кегль: заголовок – 25-30, подзаголовок – 17, основной текст – 15-12, межстрочный интервал 1,15

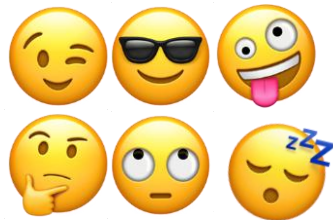
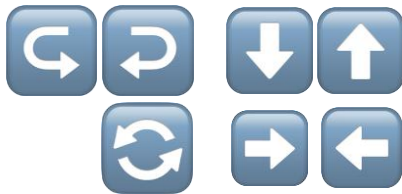
3. Используйте нашу фирменную цветовую палитру (её найдете в следующем слайде)

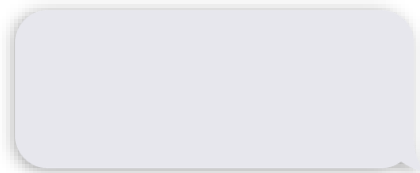
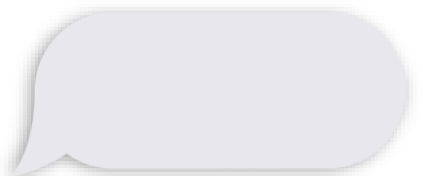
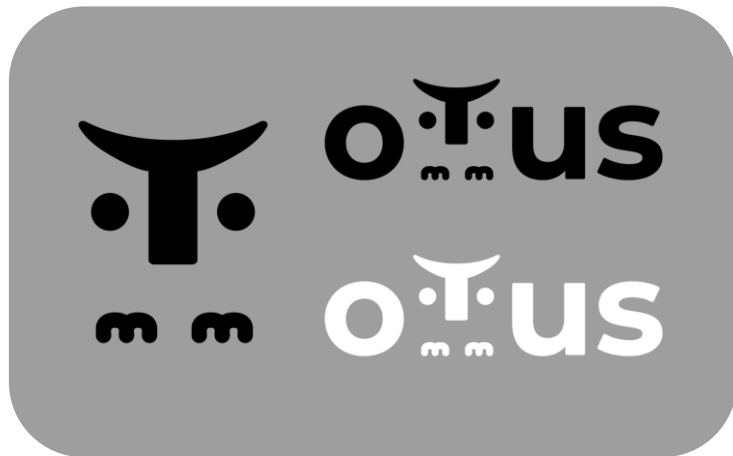
4. Визуальные материалы:

- Добавляйте релевантные изображения к текстовым блокам
- Используйте иконки или эмодзи для визуализации ключевых пунктов
- Используйте инфографику для визуализации сложной информации



качественные изображения вы можете взять [в папке](#)







**Шаблоны, которые вы
можете использовать для
рассказа о своем проекте**

Название подтемы



Определение – это

описание данных в виде набора сущностей, отношений между ними, атрибутов сущностей, их типов данных, соглашений об именовании и правил проверки целостностей



Слайды с “кнопками”

Тезис 1/Определение 1

Тезис 2 (выбирая его, словно щелкаем на него мышью, и в следующем слайде раскрываем выбранную мысль полностью) / Определение 2



Определение – описание данных в виде набора сущностей, отношений между ними, атрибутов сущностей, их типов данных, соглашений об именовании и правил проверки целостностей



Заголовок: главная мысль слайда

Подзаголовок – дополнение к основной мысли

- Источников много
- Кроме того, в той же микросервисной архитектуре у нас запрос от клиента может проходить через несколько различных микросервисов, причем в зависимости от различных аспектов, этот путь может состоять из разных узлов.

место для иллюстрации

Главная мысль слайда

- В этой статье мы будем завершать тему мониторинга и поговорим о мониторинге работы самих приложений.
- Какими средствами и как можно осуществлять сбор метрик с приложений и что такое трассировка.
- Как уже говорилось в предыдущих статьях, мы можем осуществлять сбор количественных показателей по событиям, формирующим тренды с помощью метрик.
- Кроме того, в той же микросервисной архитектуре у нас запрос от клиента может проходить через несколько различных микросервисов.
- В зависимости от различных аспектов, этот путь может состоять из разных узлов.



Главная мысль слайда



В этой статье мы будем завершать тему мониторинга и поговорим о мониторинге работы самих приложений.



Какими средствами и как можно осуществлять сбор метрик с приложений и что такое трассировка.



Как уже говорилось в предыдущих статьях, мы можем осуществлять сбор количественных показателей по событиям, формирующим тренды с помощью метрик.

Слайд со списком

- 1.
- 2.
- 3.
- 4.
- 5.

Главная мысль слайда

В случае, если мы обнаружили превышение по какому-то определенному параметру, то нам необходимо разобраться в причинах произошедшего, и здесь нам на помощь приходят журналы событий сервисов.

В логах можно найти много полезной информации касающейся проблем в работе приложения, по сути, если событие в принципе логируется, то мы можем его найти и проанализировать.

ссылка на источник кегль 11:

<https://habr.com/ru/companies/rosatom/articles/841842/>

место для иллюстрации

Слайд с 3 колонками

поменяйте иконки на
подходящие по смыслу
(слайд 100)



Над текстовым полем
могут быть иконки,
цифры, иллюстрации –
всё, на чем вы хотите
сакцентировать
внимание студентов.



Над текстовым полем
могут быть иконки,
цифры, иллюстрации –
всё, на чем вы хотите
сакцентировать
внимание студентов.



Над текстовым полем
могут быть иконки,
цифры, иллюстрации –
всё, на чем вы хотите
сакцентировать
внимание студентов.

Сравнение двух кейсов

Кейс 1

Определяющей характеристикой метрик является то, что они поддаются агрегированию, то есть, метрики похожи на атомы, которые складываются в единый логический индикатор, счетчик или гистограмму за определенный промежуток времени.

Кейс 2

В качестве примеров: текущая глубина очереди может быть смоделирована как индикатор, количество входящих HTTP-запросов может быть смоделировано как счетчик, обновления которого агрегируются простым сложением.

Определяющей характеристикой метрик является то, что они поддаются агрегированию, то есть, метрики похожи на атомы, которые складываются в единый логический индикатор, счетчик или гистограмму за определенный промежуток времени.

Определяющей характеристикой метрик является то, что они поддаются агрегированию, то есть, метрики похожи на атомы, которые складываются в единый логический индикатор, счетчик или гистограмму за определенный промежуток времени.

Определяющей характеристикой метрик является то, что они поддаются агрегированию, то есть, метрики похожи на атомы, которые складываются в единый логический индикатор, счетчик или гистограмму за определенный промежуток времени.

Заголовок: главная мысль слайда

В этой статье мы будем завершать тему мониторинга и поговорим о мониторинге работы самих приложений. Какими средствами и как можно осуществлять сбор метрик с приложений и что такое трассировка.

Как уже говорилось в предыдущих статьях, мы можем осуществлять сбор количественных показателей по событиям, формирующим тренды с помощью метрик.

Место для кейса на полях:
разместите здесь пример,
иллюстрирующий вашу
мысль, интересный факт,
ссылку на
дополнительный
источник информации
и т.д.



Мысль, объединяющая пункты списка



Описание данных в виде набора сущностей, отношений между ними, атрибутов сущностей, их типов данных, соглашений об именовании и правил проверки целостностей

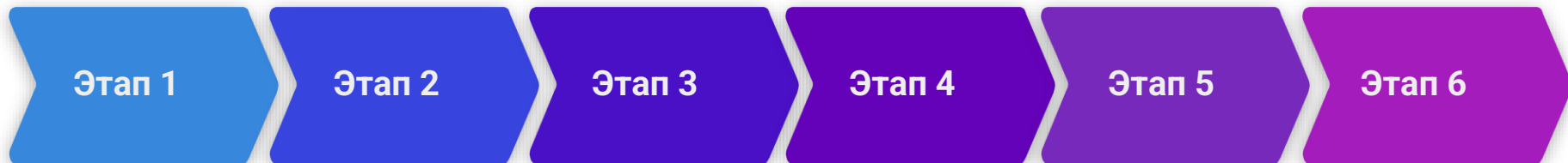


Описание данных в виде набора сущностей, отношений между ними, атрибутов сущностей, их типов данных, соглашений об именовании и правил проверки целостностей



Описание данных в виде набора сущностей, отношений между ними, атрибутов сущностей, их типов данных, соглашений об именовании и правил проверки целостностей

Процесс через этапы



Процесс через этапы

Этап 1

Описание данных в виде набора сущностей, отношений между ними, атрибутов сущностей, их типов данных, соглашений об именовании и правил проверки целостностей

Этап 2

Описание данных в виде набора сущностей, отношений между ними, атрибутов сущностей, их типов данных, соглашений об именовании и правил проверки целостностей

Этап 3

Описание данных в виде набора сущностей, отношений между ними, атрибутов сущностей, их типов данных, соглашений об именовании и правил проверки целостностей

Этап 4

Описание данных в виде набора сущностей, отношений между ними, атрибутов сущностей, их типов данных, соглашений об именовании и правил проверки целостностей

Процесс через этапы

Этап 1

Описание данных в виде набора сущностей, отношений между ними, атрибутов сущностей, их типов данных, соглашений об именовании и правил проверки целостностей

Этап 2

Описание данных в виде набора сущностей, отношений между ними, атрибутов сущностей, их типов данных, соглашений об именовании и правил проверки целостностей

Этап 3

Описание данных в виде набора сущностей, отношений между ними, атрибутов сущностей, их типов данных, соглашений об именовании и правил проверки целостностей

Этап 4

Описание данных в виде набора сущностей, отношений между ними, атрибутов сущностей, их типов данных, соглашений об именовании и правил проверки целостностей

Этап 5

Описание данных в виде набора сущностей, отношений между ними, атрибутов сущностей, их типов данных, соглашений об именовании и правил проверки целостностей

Процесс через этапы

Этап 1

Описание данных в виде набора сущностей, отношений между ними, атрибутов сущностей, их типов данных, соглашений об именовании и правил проверки целостностей

Этап 2

Описание данных в виде набора сущностей, отношений между ними, атрибутов сущностей, их типов данных, соглашений об именовании и правил проверки целостностей

Этап 3

Описание данных в виде набора сущностей, отношений между ними, атрибутов сущностей, их типов данных, соглашений об именовании и правил проверки целостностей

Этап 4

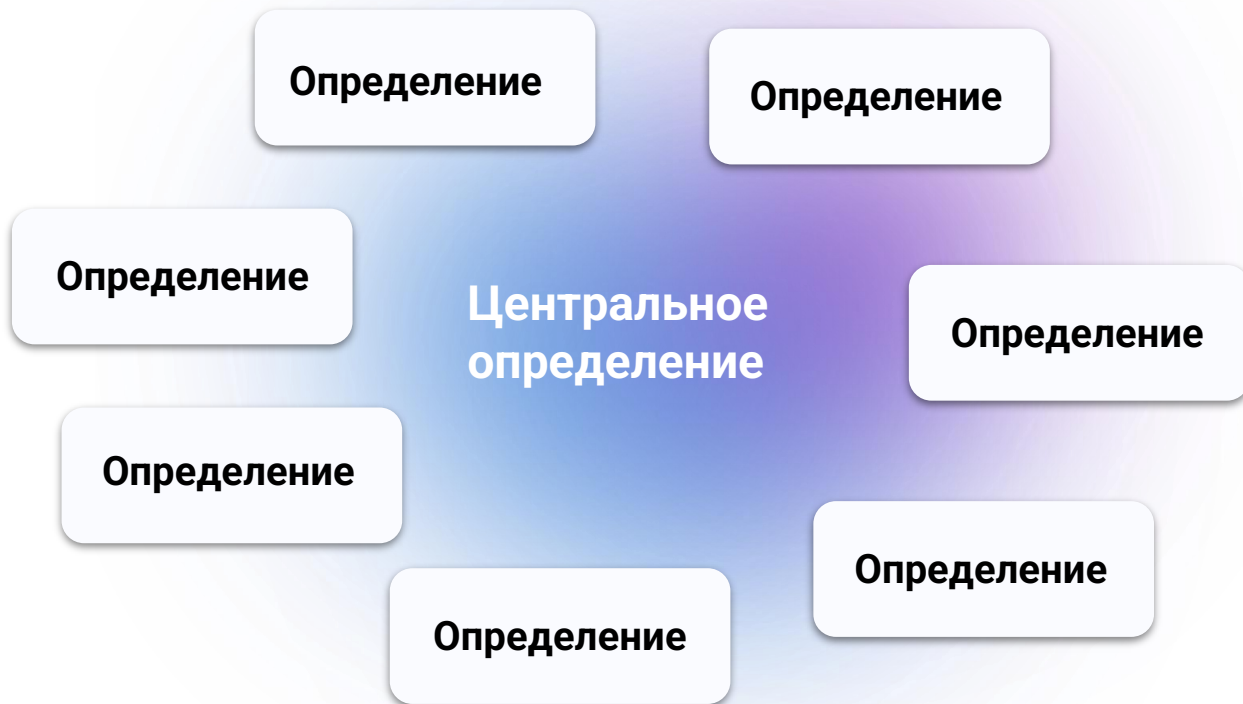
Описание данных в виде набора сущностей, отношений между ними, атрибутов сущностей, их типов данных, соглашений об именовании и правил проверки целостностей

Этап 5

Описание данных в виде набора сущностей, отношений между ними, атрибутов сущностей, их типов данных, соглашений об именовании и правил проверки целостностей



Определения, которые нужно сгруппировать



Слайд с кодом 1

```
int **a; // указатель на указатель на строку элементов
int i, j, n, m;
system("chcp 1251");
system("cls");
printf("Введите количество строк: ");
scanf("%d", &n);
printf("Введите количество столбцов: ");
scanf("%d", &m);
// Выделение памяти под указатели на строки
a = (int**)malloc(n * sizeof(int*));
// Ввод элементов массива
for (i = 0; i < n; i++) // цикл по строкам
{
    // Выделение памяти под хранение строк
    a[i] = (int*)malloc(m * sizeof(int));
    for (j = 0; j < m; j++) // цикл по столбцам
    {
        printf("a[%d][%d] = ", i, j);
        scanf("%d", &a[i][j]);
    }
}
```

(n *
sizeof(int*))

Слайд с кодом 2

```
int **a; // указатель на указатель на строку элементов
int i, j, n, m;
system("chcp 1251");
system("cls");
printf("Введите количество строк: ");
scanf("%d", &n);
printf("Введите количество столбцов: ");
scanf("%d", &m);
// Выделение памяти под указатели на строки
a = (int**)malloc (n * sizeof(int*));
// Ввод элементов массива
for (i = 0; i<n; i++) // цикл по строкам
{
    // Выделение памяти под хранение строк
    a[i] = (int*)malloc(m * sizeof(int));
    for (j = 0; j<m; j++) // цикл по столбцам
    {
        printf("a[%d][%d] = ", i, j);
        scanf("%d", &a[i][j]);
    }
}
```



Слайд с кодом 3

```
int **a; // указатель на указатель на строку элементов
int i, j, n, m;
system("chcp 1251");
system("cls");
printf("Введите количество строк: ");
scanf("%d", &n);
printf("Введите количество столбцов: ");
scanf("%d", &m);
// Выделение памяти под указатели на строки
a = (int**)malloc(n * sizeof(int*));
// Ввод элементов массива
for (i = 0; i<n; i++) // цикл по строкам
{
    // Выделение памяти под хранение строк
    a[i] = (int*)malloc(m * sizeof(int));
    for (j = 0; j<m; j++) // цикл по столбцам
    {
        printf("a[%d][%d] = ", i, j);
        scanf("%d", &a[i][j]);
    }
}
```

Слайд с кодом и текстом

1. Тезис 1
2. Тезис 2
3. Тезис 3

```
int **a; // указатель на строку
int i, j, n, m;
system("chcp 1251");
system("cls");
printf("Введите количество строк: ");
scanf("%d", &n);
printf("Введите количество столбцов: ");
scanf("%d", &m);
// Выделение памяти под хранение строк
```

Слайд с таблицей

Название столбца	Название столбца	Название столбца
Текст нежирным шрифтом	Текст нежирным шрифтом	Текст нежирным шрифтом

Слайд с таблицей

	Название столбца	Название столбца	Название столбца
1.	Текст нежирным шрифтом	Текст нежирным шрифтом	Текст нежирным шрифтом
2.			
3.			
4.			
5.			

**Единственно верного
решения не существует.
Хотя существует
много неверных.**

Дэвид Акин, NASA